

INFORME TÉCNICO

Laboratorio VIII — Sistemas Operativos y Seguridad III (SOS III)

*Despliegue de Contenedores Docker en AWS EC2 sobre RHEL 10:
Nginx, Portainer y WordPress con MySQL*

Estudiante: Victor José De Peña Bernard

Matrícula: 2023-0506

Repositorio: github.com/vitin0611/LABORATORIO-VIII---SOS-III

Santo Domingo, República Dominicana — Julio 2026

1. Introducción

El presente informe documenta la ejecución de tres prácticas de laboratorio correspondientes a la asignatura Laboratorio VIII — Sistemas Operativos y Seguridad III (SOS III), enfocadas en la contenerización de aplicaciones mediante Docker sobre una instancia de Amazon Web Services (AWS) EC2 con sistema operativo Red Hat Enterprise Linux 10 (RHEL 10). Las prácticas abarcan desde la instalación del motor de Docker y el despliegue de un servidor web estático, hasta la administración gráfica de contenedores con Portainer y la orquestación de una pila multiservicio con Docker Compose para un sitio WordPress respaldado por una base de datos MySQL.

El código fuente, los archivos de configuración y los comandos utilizados se encuentran disponibles en el repositorio del proyecto.

2. Objetivos

- Instalar y configurar el motor de Docker CE en un servidor RHEL 10 alojado en AWS EC2.
- Desplegar un contenedor Nginx con contenido web persistente mediante volúmenes montados desde el host.
- Configurar el firewall del sistema (firewalld) para permitir el tráfico en los puertos requeridos por cada servicio.
- Desplegar Portainer CE como interfaz gráfica de administración remota de contenedores Docker.
- Orquestar, mediante Docker Compose, una pila compuesta por WordPress y MySQL con persistencia de datos y variables de entorno.
- Configurar los grupos de seguridad (Security Groups) de AWS EC2 para permitir el acceso público a cada servicio desplegado.
- Verificar el correcto funcionamiento de cada despliegue mediante pruebas locales (curl) y acceso remoto vía navegador.

3. Entorno y herramientas utilizadas

- Proveedor cloud: Amazon Web Services (AWS), instancia EC2.
- Sistema operativo del servidor: Red Hat Enterprise Linux 10 (RHEL 10).
- Motor de contenedores: Docker CE, Docker Compose Plugin, containerd.io.
- Firewall del sistema: firewalld.
- Acceso remoto: SSH mediante clave privada (PWS_LINUX.pem), usuario ec2-user.
- Imágenes de contenedor: nginx, portainer/portainer-ce, mysql:8.0, wordpress:latest.
- Dirección IP pública utilizada durante las prácticas: 34.201.82.37.

4. Práctica 1 — Instalación de Docker y despliegue de Nginx

Esta práctica cubre la instalación desde cero del motor de Docker sobre RHEL 10 y el despliegue de un servidor web Nginx cuyo contenido se sirve desde un volumen persistente en el host, identificando al estudiante mediante una página index.html personalizada.

4.1 Instalación del motor de Docker

Se agregó el repositorio oficial de Docker CE para RHEL y se instalaron el motor, el CLI, containerd y los plugins de buildx y compose:

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

Dado que RHEL 10 introduce cambios en el manejo de nftables/iptables, fue necesario deshabilitar la gestión directa de iptables por parte de Docker, ajustando /etc/docker/daemon.json, y habilitar tanto containerd como el propio servicio Docker:

```
sudo mkdir -p /etc/docker
sudo bash -c 'cat <<EOF > /etc/docker/daemon.json
{ "iptables": false }
EOF'
sudo systemctl enable --now containerd
sudo systemctl daemon-reload
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
newgrp docker
```

Añadir el usuario al grupo docker permite ejecutar comandos del cliente sin anteponer sudo en cada instrucción.

4.2 Configuración del firewall (firewalld)

Se habilitó de forma permanente el puerto 8888/tcp, requerido para exponer el contenedor Nginx:

```
sudo firewall-cmd --permanent --add-port=8888/tcp
sudo firewall-cmd --reload
```

4.3 Volumen persistente y contenido web

Se creó el directorio /home/website en el host y un archivo index.html con los datos del estudiante, el cual se monta posteriormente dentro del contenedor:

```
sudo mkdir -p /home/website
sudo bash -c 'cat <<EOF > /home/website/index.html
<!DOCTYPE html>
<html><head><title>Practica Docker</title></head>
<body>
  <h1>Victor Jose De Peña Bernard</h1>
```

```
<h2>2023-0506</h2>
</body></html>
EOF'
```

4.4 Despliegue del contenedor Nginx

Se descargó la imagen oficial de Nginx y se desplegó el contenedor mapeando el puerto 8888 del host al 80 del contenedor, montando el volumen del host con la etiqueta :z requerida por SELinux para permitir el acceso al contenido desde el contenedor:

```
docker pull nginx
docker run -d \
  --name mi-nginx \
  -p 8888:80 \
  -v /home/website:/usr/share/nginx/html:z \
  nginx
```

4.5 Verificación

La verificación se realizó en dos niveles: localmente desde la propia instancia mediante curl, y de forma remota accediendo desde un navegador a la IP pública del servidor:

```
curl http://127.0.0.1:8888
# Verificación remota: http://34.201.82.37:8888
```

El resultado esperado es la carga correcta de la página HTML con el nombre y la matrícula del estudiante, confirmando que Docker, el volumen montado y la regla de firewall funcionan correctamente en conjunto.

5. Práctica 2 — Despliegue de Portainer

Esta práctica implementa Portainer CE, una interfaz web de administración que permite gestionar contenedores, imágenes, volúmenes y redes Docker de forma gráfica, evitando el uso exclusivo de la línea de comandos.

5.1 Conexión SSH a la instancia

El acceso a la instancia EC2 se realiza mediante clave privada, ajustando previamente los permisos del archivo .pem en entornos Linux/Mac:

```
chmod 400 PWS_LINUX.pem
ssh -i PWS_LINUX.pem ec2-user@34.201.82.37
```

5.2 Volumen persistente para Portainer

Se creó un volumen Docker dedicado para preservar la configuración y los datos de Portainer entre reinicios del contenedor:

```
docker volume create portainer_data
```

5.3 Despliegue del contenedor

Portainer se desplegó exponiendo el puerto 9000 (HTTP) y 9443 (HTTPS), montando el socket de Docker del host para permitirle administrar el propio motor Docker de la máquina, junto al volumen persistente creado previamente:

```
docker run -d \
  -p 9000:9000 \
  -p 9443:9443 \
  --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
```

El montaje del socket `/var/run/docker.sock` es lo que otorga a Portainer control administrativo sobre el motor Docker del host; por tratarse de un privilegio elevado, el acceso a la interfaz debe protegerse con credenciales robustas desde el primer inicio.

5.4 Configuración de seguridad en AWS (Security Group)

Se habilitaron las siguientes reglas de entrada en el Security Group de la instancia EC2:

Tipo	Puerto	Origen	Descripción
Custom TCP	9000	0.0.0.0/0	Acceso web HTTP a Portainer
Custom TCP	9443	0.0.0.0/0	Acceso web HTTPS a Portainer

5.5 Verificación y acceso

```
docker ps
curl -I http://127.0.0.1:9000
# Acceso remoto:
# HTTP: http://34.201.82.37:9000
# HTTPS: https://34.201.82.37:9443
```

En el primer acceso a la interfaz web, Portainer solicita la definición de la contraseña del usuario administrador (admin), paso indispensable antes de poder visualizar y gestionar los recursos Docker del host.

6. Práctica 3 — WordPress y MySQL con Docker Compose

Esta práctica introduce la orquestación de múltiples contenedores mediante Docker Compose, desplegando una pila completa de WordPress con su base de datos MySQL en servicios independientes pero interconectados a través de una red interna de Docker.

6.1 Optimización de memoria swap

Dado que las instancias EC2 de bajos recursos pueden bloquearse por consumo elevado de RAM al inicializar MySQL y WordPress simultáneamente, se configuró una partición swap de 2 GB, haciéndola persistente en `/etc/fstab`:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
free -m
```

6.2 Archivo docker-compose.yml

Se definieron dos servicios: db (MySQL 8.0, con base de datos, usuario y contraseña dedicados, y volumen persistente db_data) y wordpress (imagen oficial, expuesta en el puerto 8080 del host, configurada para conectarse al servicio db mediante las variables de entorno WORDPRESS_DB_*):

```
mkdir -p ~/practica3-wordpress && cd ~/practica3-wordpress
```

```
services:
  db:
    image: mysql:8.0
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: wordpress_root_pass
      MYSQL_DATABASE: wordpress_db
      MYSQL_USER: wordpress_user
      MYSQL_PASSWORD: wordpress_pass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress-app
    restart: always
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress_user
      WORDPRESS_DB_PASSWORD: wordpress_pass
      WORDPRESS_DB_NAME: wordpress_db
    depends_on:
      - db

volumes:
  db_data:
```

La directiva `depends_on` garantiza que el contenedor `db` se inicie antes que `wordpress`, mientras que Docker Compose crea automáticamente una red interna que permite a WordPress resolver el nombre de servicio `db` como si fuera un host DNS.

6.3 Despliegue y gestión de la pila

```
docker compose up -d
docker ps
docker compose logs -f
curl -I http://127.0.0.1:8080
```

6.4 Configuración de seguridad en AWS (Security Group)

Tipo	Puerto	Origen	Descripción
Custom TCP	8080	0.0.0.0/0	Acceso web a WordPress

6.5 Verificación y acceso

Tras confirmar mediante `docker ps` que ambos contenedores se encuentran en estado "Up", y validar la respuesta HTTP local con `curl`, se completó la instalación inicial de WordPress accediendo desde el navegador a la IP pública en el puerto 8080:

```
http://34.201.82.37:8080
```

7. Análisis de seguridad

- En las tres prácticas, el acceso administrativo remoto se realiza exclusivamente mediante autenticación por clave SSH (.pem), evitando el uso de contraseñas.
- Los Security Groups de AWS se configuraron de forma específica por puerto y servicio, evitando exposiciones innecesarias de otros puertos del sistema.
- El montaje del socket de Docker en el contenedor de Portainer constituye un punto sensible: otorga privilegios equivalentes a root sobre el host, por lo que el acceso a su interfaz debe restringirse con credenciales fuertes y, idealmente, HTTPS.
- Las credenciales de la base de datos MySQL en la Práctica 3 se definieron mediante variables de entorno en texto plano dentro del `docker-compose.yml`; en un entorno de producción se recomienda migrar estos valores a un archivo `.env` excluido del control de versiones, o a un gestor de secretos.
- El uso de volúmenes nombrados (`db_data`, `portainer_data`) asegura la persistencia de datos críticos ante la eliminación o recreación de los contenedores.
- La bandera `:z` en el montaje de volúmenes de la Práctica 1 es necesaria en RHEL debido a SELinux, ya que ajusta automáticamente el contexto de seguridad del contenido compartido entre el host y el contenedor.

8. Conclusiones

El desarrollo de las tres prácticas permitió consolidar el flujo completo de trabajo con contenedores Docker sobre un servidor RHEL 10 en la nube: desde la instalación y configuración inicial del motor, pasando por el despliegue de servicios individuales gestionados manualmente (Nginx, Portainer), hasta la orquestación de una arquitectura multiservicio con Docker Compose (WordPress y MySQL). Se evidenció la importancia de coordinar correctamente el firewall del sistema operativo (firewalld) con los Security Groups de AWS para exponer únicamente los puertos necesarios, así como la relevancia de la persistencia de datos mediante volúmenes y de las políticas de reinicio automático (restart: always) para garantizar la disponibilidad de los servicios ante fallos o reinicios de la instancia.

9. Referencias

Repositorio del proyecto: github.com/vitin0611/LABORATORIO-VIII---SOS-III

Documentación oficial de Docker: docs.docker.com

Documentación oficial de Portainer CE: docs.portainer.io

Documentación oficial de WordPress Docker Official Image: hub.docker.com/_/wordpress